

Bubble Bobble Vibe Coding Lab

Prompt & Response Evidence

Three modifications — prompts followed by responses

Prompt 1 — Distance Squared Bug

ab cad9c-d7e8-83ee-9a9b-e9a590c0fae3

Share

I am completing a Vibe Coding assignment on this Bubble Bobble Python/Pygame project.

Please inspect the existing `game.py` code and identify the deliberate bug related to trapping an enemy inside a bubble.

Fix ONLY that bug.

The code uses `pygame Vector2 distance_squared_to()`. Check whether the distance comparison is mathematically correct.

Do not redesign the game.
Do not add any new features yet.
Do not modify unrelated code.

After making the change, explain:

1. What the bug was
2. Why it was incorrect
3. What exact change was made.

I inspected the uploaded `game.py`. The **distance comparison is already mathematically correct**, so I did **not** change unrelated code or introduce a new modification.

1. What the bug was

The potential bug with `distance_squared_to()` would be comparing the **squared distance** directly against an ordinary distance, for example:

+ Ask ChatGPT

Think

Response 1 — Bug Explanation and Fix

Share ...

1. What the bug was

The potential bug with `distance_squared_to()` would be comparing the **squared distance** directly against an ordinary distance, for example:

```
</> Python ↗ 📄 ▶ Run  
bubble.pos.distance_squared_to(enemy.center) < 24
```

That would be incorrect.

2. Why

`distance_squared_to()` returns:

$$distance^2$$

So if the intended trapping radius is **24 pixels**, the correct comparison is:

$$distance^2 < 24^2$$

Your current code does exactly that:

```
</> Python ↗ 📄 ▶ Run  
if bubble.pos.distance_squared_to(enemy.center) < 24 * 24:
```

This appears at the enemy-trapping logic. </> game_modified

3. Exact change made

No change was made, because the uploaded file `alr...` contains the correct fix:

+ Ask ChatGPT 🧠 Think 🎤 🟩

Prompt 2 — Bubble Tint

/6abcad9c-d7e8-83ee-9a9b-e9a590c0fae3

Share

I am continuing my Bubble Bobble Vibe Coding assignment.

The first bug has already been fixed. Do NOT change that fix.

Now implement ONLY the bubble_tint(bubble) feature.

Requirements:

- Implement the existing bubble_tint(bubble) function.
- Use the bubble's existing life value.
- Return an (r, g, b) RGB tuple when a tint should be applied.
- Make bubbles visually change color as their remaining life decreases.
- Return None when the default bubble color should be used.
- Keep the implementation simple.
- Do not redesign the game.
- Do not modify the fruit or bonus-life functions yet.
- Do not change unrelated gameplay code.

After making the change, explain what was changed.

Worked for 8s

I checked the uploaded `game.py`. `bubble_tint(bubble)` is already implemented correctly, so I did not change anything else.

The current implementation is:

</> Python

↔

📄

▶ Run

+ Ask ChatGPT

🧠 Think

🔊

Response 2 — Bubble Tint Implementation

I checked the uploaded `game.py`. `bubble_tint(bubble)` is already implemented correctly, so I did not change anything else.

The current implementation is:

`</>` Python



Run

```
def bubble_tint(bubble):
    """Return an (r, g, b) colour for a bubble, or None for the default."""

    if bubble.life > 5:
        return None
    elif bubble.life > 2:
        return (255, 220, 80)
    else:
        return (255, 80, 80)
```

`</>` game_modified

What it does

- Life > 5 seconds: `None` → uses the normal/default bubble color.
- Life between 2 and 5 seconds: `(255, 220, 80)` → yellow tint.
- Life ≤ 2 seconds: `(255, 80, 80)` → red tint.

So the bubble changes color as its remaining life decreases, exactly as required.

- ✓ First `distance_squared_to()` bug remains untouched.
- ✓ `bubble_tint()` is simple and uses the existing `bubble.life`.
- ✓ No fruit, bonus-life, or unrelated gameplay code was modified.



Prompt 3 — Fruit Collection

Share



I am continuing my Bubble Bobble Vibe Coding assignment.

The following two tasks are already completed and working:

1. Fixed the `distance_squared_to()` bubble-trapping bug.
2. Implemented `bubble_tint(bubble)`.

Do NOT modify either of these.

Now inspect the existing `game.py` and implement ONLY the `on_fruit_collected(fruit)` feature.

Requirements:

- Keep the existing `on_fruit_collected(fruit)` function.
- Add a simple visible fruit-collection feature.
- Maintain a fruit collection count.
- Display the count in the existing HUD.
- Increase the count whenever the player collects a fruit.
- Reset the count when the game is reset.
- Do not break the existing score or lives system.

...

Show more

Worked for 19s

I inspected the existing `Game` class, fruit flow, reset logic, and HUD.

The requested fruit-count feature is **already implemented** in the uploaded `game.py`:

+ Ask ChatGPT

Think



Response 3 — Fruit Collection Implementation

...

Show more ▾

Worked for 19s

I inspected the existing `Game` class, fruit flow, reset logic, and HUD.

The requested fruit-count feature is **already implemented in the uploaded `game.py`** :

- `fruit_count` is initialized/reset to `0` in `Game.reset()`. `</> game_modified`
- It increases when a fruit is collected. `</> game_modified`
- The count is displayed in the existing HUD. `</> game_modified`
- The existing score and lives logic remains separate.

The only thing left in `on_fruit_collected(fruit)` is `pass` , but that callback isn't needed for the required count functionality because the count is already handled directly in the fruit-collection flow. `</> game_modified`

So I **did not add unnecessary behavior or modify the already-working implementation**. Your current file already satisfies the requirements for this feature, while leaving both the `distance_squared_to()` fix and `bubble_tint()` untouched.

📄 📌 ⬆️ 🔊 🔄 ... 📖

ChatGPT can make mistakes. Check important info.

+ | Ask ChatGPT

🧠 Think

